WORKSHOP

Build Your AI Project Manager

An agentic assistant with CrewAI and local open models — hands-on, from first agent to a virtual office

Felix Yustian Setiono · Senior AI / Computer Vision Engineer

A hands-on MurzEdu session · ~2 hours · Python + a laptop that can run Ollama



Felix Yustian Setiono

Senior AI / Computer Vision (CV) Engineer
AI / ML Systems Architect

Semarang, Central Java, Indonesia
felixyustian@gmail.com
linkedin.com/in/felixsetiono

CURRENTLY

Sigmawave AI Singapore — Senior AI/ML & CV Engineer & CV Team Lead
V-TEKI Indonesia — AI Architect & Technical Lead

PROFILE

10+ years turning AI into working business systems — from computer vision and predictive models to LLM-powered, agentic automation. Bridges real engineering with the Indonesian market: regulation, language, and SME operations.

CORE SKILLS

- **LLM & Agentic Systems:** CrewAI, LangChain / LangGraph, MCP, multi-agent
- **Computer Vision & Edge AI:** YOLO, TensorRT, NVIDIA Jetson, DeepStream
- **Machine Learning & MLOps:** Python, PyTorch, forecasting, FastAPI, Docker
- **Document AI:** OCR, KTP / ID extraction, governed LLM extraction

SELECTED HIGHLIGHTS

- 5-agent business assistant for Indonesia's 65M SMEs (OpenClaw Agenthon 2026 winner)
- Predictive maintenance for industry — flag failures before they happen (MachineGuard)
- Governed AI for OJK-regulated decisions — auditable and deployable (PRANATA)
- Document / KYC automation — IDs & invoices read automatically, PII-safe

CREDENTIALS & RECOGNITION

M.Eng., ITB · Professional Engineer (Ir.) · Google Cloud GenAI Leader · Published researcher (Scholar h-index 4) · IEEE / PII / KORIKA member

What we'll build today

1. The goal — an agent crew that helps run your projects
2. How agents actually work: LLM + tools + a loop
3. Set up CrewAI on local open models (Ollama)
4. Build the PM assistant — step by step, in code
5. Intermezzo — a virtual office of agents
6. Make it reliable, observable, and safe

What we're building: your AI PM

ai-pm — python pm_crew.py

```
$ python pm_crew.py --brief brief.md
[Planner]  breaking the brief into tasks...
[Risk]     top 3 risks, each with a mitigation
[Writer]   kickoff status update drafted

=== RESULT: 10 tasks planned ===
```

TASK	OWNER	DAYS
Meet with design lead to discuss onboarding…	PM	2
Define and document the new 3-step signup p…	PM	3
Create wireframes and mockups of the new si…	Designer	4
Collaborate on implementing design changes,…	Engineer 1	5

```
saved to project_state.json
```

WHAT IT DOES

— Turns a project brief into tasks, owners, and estimates

— Flags risks and drafts a status update

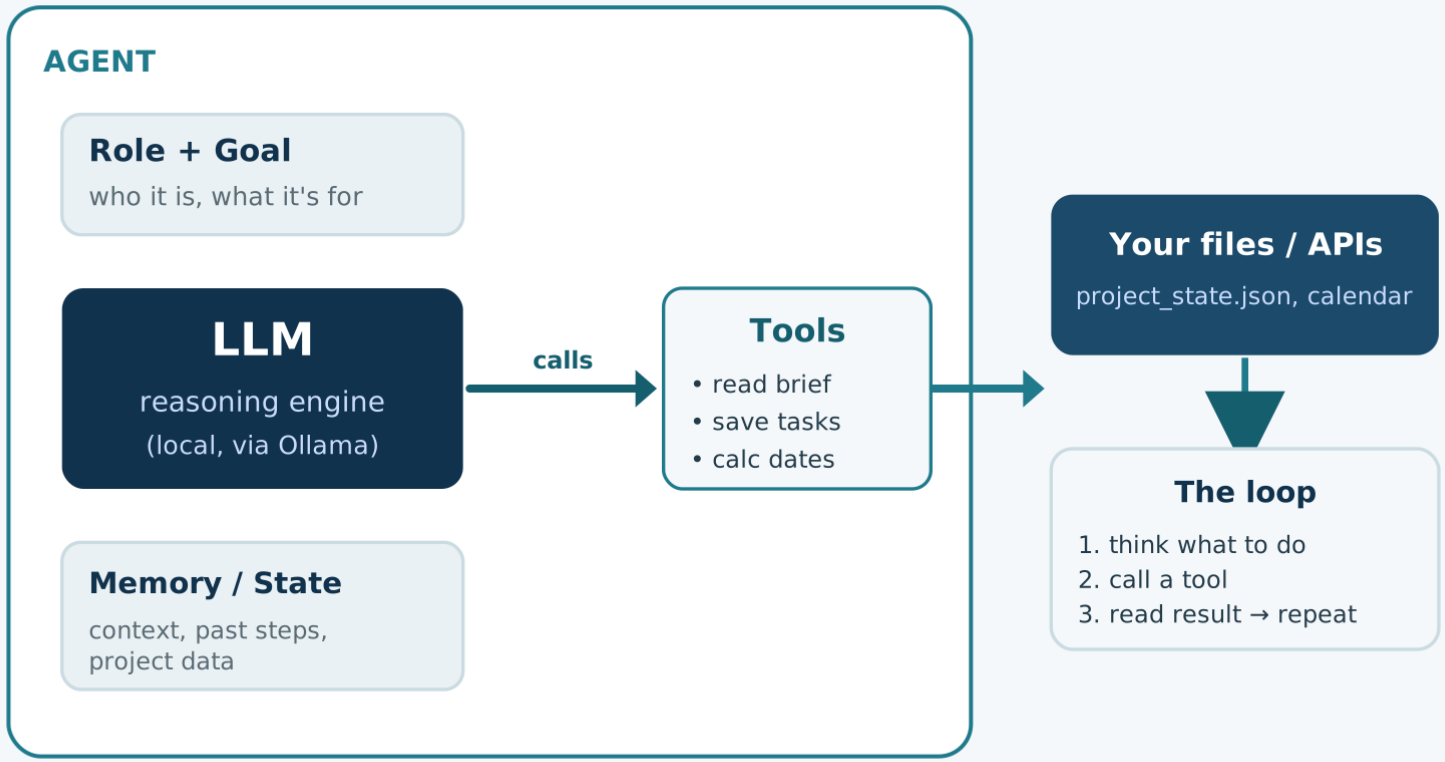
— Runs locally — your data never leaves the machine

And you'll build it in code, today.

01 How Agents Work

LLM + tools + a loop — the moving parts

Anatomy of an agent

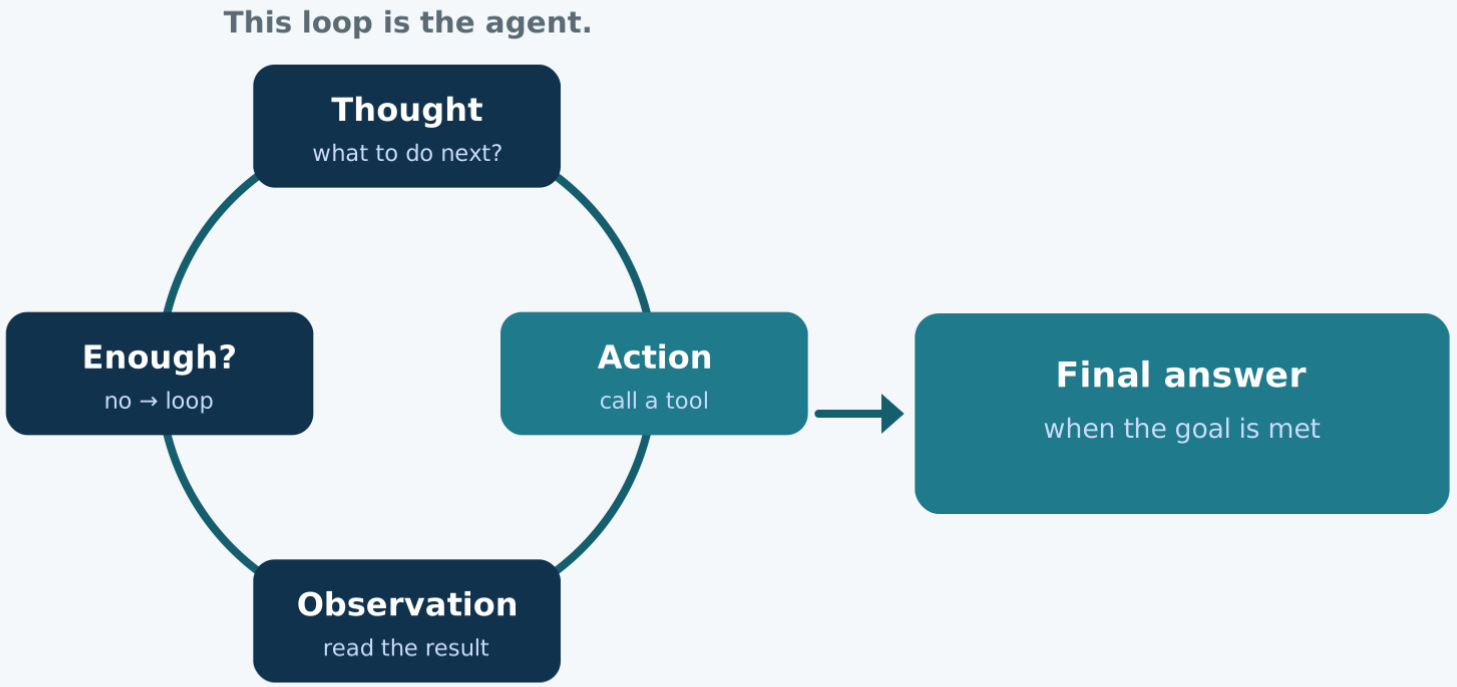


FOUR PARTS

- An LLM that reasons
- Tools it can call (plain functions)
- Memory / state it carries
- A loop that ties them together

Strip the hype: an agent is a loop around a model.

The loop: Reason → Act → Observe (ReAct)



EACH TURN

- Think about the next step
- Act — call a tool
- Observe the result
- Repeat until the goal is met

CrewAI runs this loop for you.

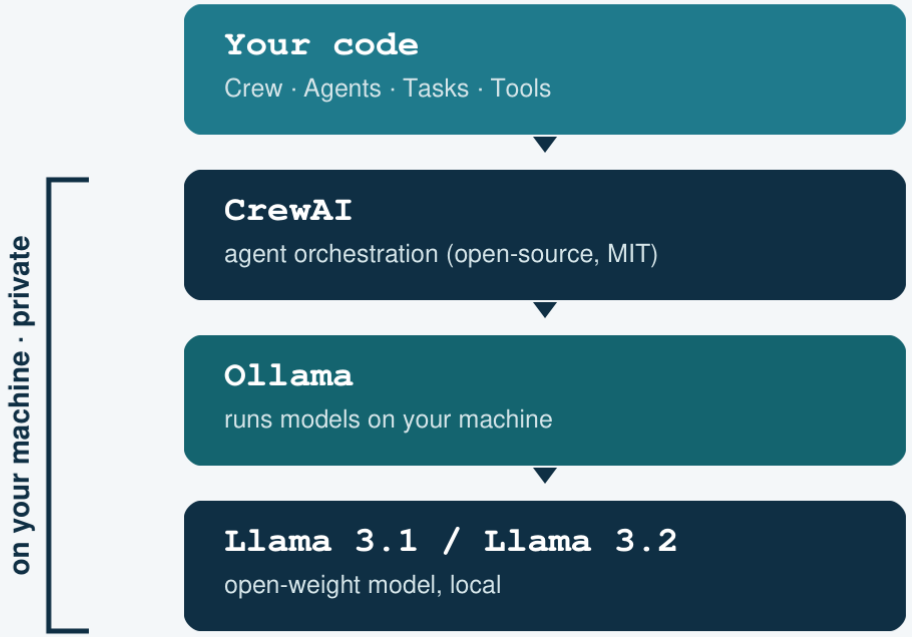
One agent, or a crew?

- One agent is enough for a single, focused job.
- **Hard jobs split better across specialists —** a planner, a researcher, a writer.
- A crew coordinates them: sequential (a pipeline) or hierarchical (a manager delegates).
- More agents = more capability, but more cost, latency, and nondeterminism.

RULE OF THUMB

Start with one agent. Add roles only when one agent clearly struggles.

Open-source, and running on your machine



Why local?

- data stays in-house
- no per-token cost
- full control
- works offline

THE STACK

— CrewAI orchestrates the agents

— CrewAI talks to Ollama directly

— Ollama runs open models locally

— Llama 3.1 / Llama 3.2 — open weights

No API keys. No data leaving your laptop.

02

Build the PM Assistant

A brief goes in; a plan comes out

Setup — Ollama + CrewAI

Install once, pull a model, add the framework. Windows: .venv\Scripts\activate

```
# 1. install Ollama (ollama.com), then pull open models
ollama pull llama3.1          # ~4.7 GB, runs fully local
ollama pull llama3.2:3b       # ~2 GB, for weaker laptops

# 2. a clean Python 3.10-3.13 environment + the framework
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. sanity check: is everything ready?
python check_setup.py
```

→ **Pinned to crewai==1.15.35 so the session is reproducible across laptops.**

Your first agent

Agent + Task + Crew, pointed at the local model.

```
from crewai import Agent, Task, Crew, Process, LLM

# point CrewAI at the LOCAL model served by Ollama
llm = LLM(model="ollama/llama3.1",
          base_url="http://localhost:11434")

planner = Agent(
    role="Project Planner",
    goal="Turn a project brief into a clear task list",
    backstory="A sharp PM who makes vague briefs actionable.",
    llm=llm, verbose=True,
)

task = Task(
    description="Break down this brief into tasks:\n{brief}",
    expected_output="A numbered task list with an owner each",
    agent=planner,
)
```

→ **One agent, one task, a crew to run it — a working agent in ~20 lines.**

Give it a tool

Tools are just Python functions the agent may choose to call.

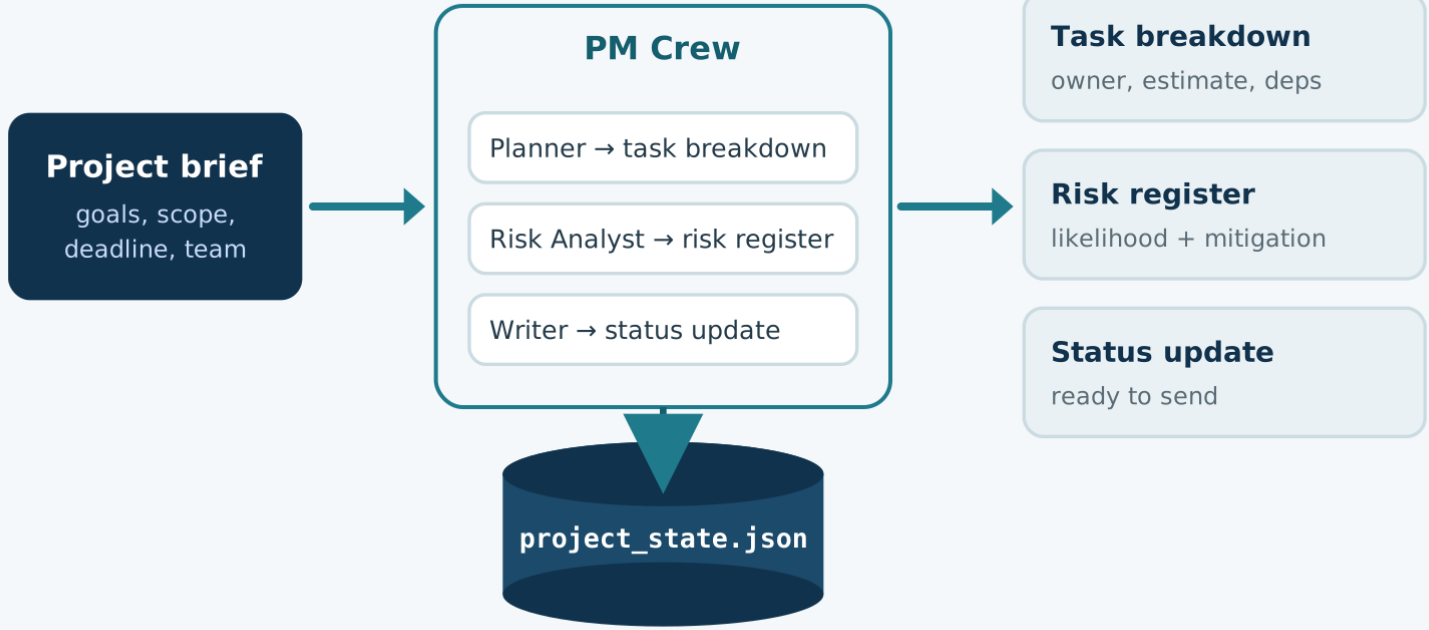
```python
from crewai.tools import tool
import json

@tool("save_tasks", max_usage_count=1)
def save_tasks(tasks_json: str) -> str:
    "Save a JSON list of task titles to tasks.json."
    data = json.loads(tasks_json)
    with open("tasks.json", "w") as f:
        json.dump(data, f, indent=2)
    return f"saved {len(data)} tasks"

# the agent may now decide to call this tool
planner.tools = [save_tasks]
```

→ **The model decides whether to call it. Run tool_demo.py a few times: sometimes it won't.**

The PM assistant: the shape



THE FLOW

- Brief in → the PM crew
- Planner, Risk Analyst, Writer
- Out: tasks, risks, a status update
- Shared state in project_state.json

Three specialists, one shared state file.

Typed output, not prose

Make the planner return structured data the next step can trust.

```
from pydantic import BaseModel
from typing import List

class TaskItem(BaseModel):
    title: str
    owner: str
    estimate_days: int

class Plan(BaseModel):
    tasks: List[TaskItem]

plan_task = Task(
    description="Break the brief into tasks:\n{brief}",
    expected_output="A Plan object",
    agent=planner,
    output_pydantic=Plan,     # validated, typed output
)
```

→ **A Pydantic schema means no regex-parsing of prose — the output is validated and typed.**

Add two specialists

A risk analyst and a status writer, each with one job.

```
risk = Agent(
    role="Risk Analyst",
    goal="Flag the top risks, with one mitigation each",
    backstory="A cautious PM who has watched projects slip.",
    llm=llm)

writer = Agent(
    role="Status Writer",
    goal="Draft a short status update for stakeholders",
    backstory="Writes clear, calm updates and never invents facts.",
    llm=llm)

risk_task = Task(description="Review the plan. List the top risks.",
                 expected_output="A risk register", agent=risk,
                 context=[plan_task])

status_task = Task(description="Write a kickoff update. Work has NOT started: claim no progress.",
                   expected_output="At most 5 lines", agent=writer,
                   context=[plan_task, risk_task])
```

→ **One agent = one responsibility. Keep roles narrow and goals sharp.**

Run the PM crew

Wire the three agents into one sequential crew and run it.

```
from pathlib import Path

crew = Crew(
    agents=[planner, risk, writer],
    tasks=[plan_task, risk_task, status_task],
    process=Process.sequential,     # planner -> risk -> writer
    verbose=True,
)

result = crew.kickoff(inputs={"brief": open("brief.md").read()})
print(result)      # the final task's output: the status update

# save shared state from Python, so it can't depend on the model's choice
plan = result.tasks_output[0].pydantic
Path("project_state.json").write_text(plan.model_dump_json(indent=2))
```

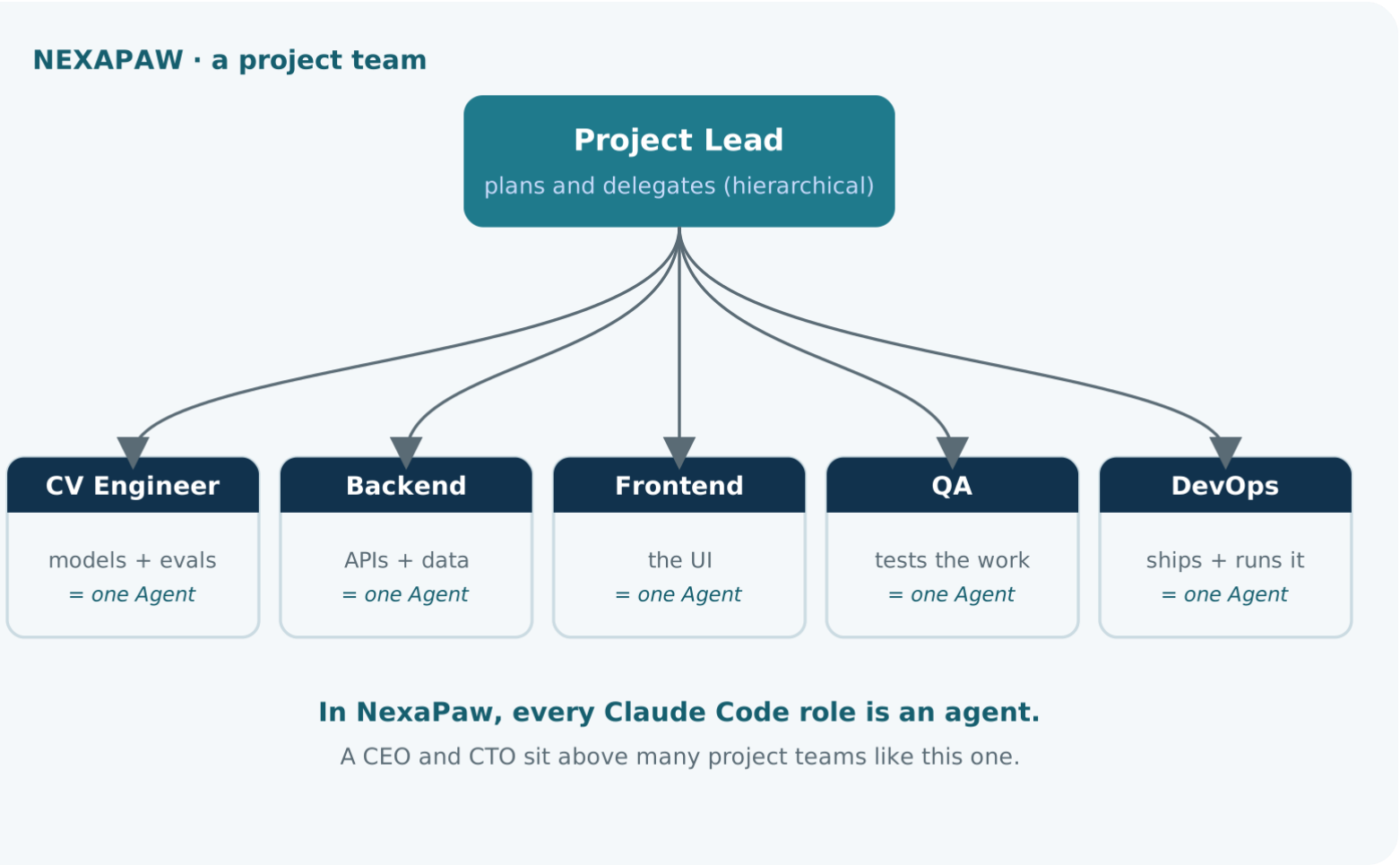
→ **Run it on a real brief. That's your AI project manager — in about 40 lines.**



Intermezzo · The Virtual Office

When one assistant becomes a team

NexaPaw: a real agentic office



THE IDEA

— Each Claude Code role is an agent — a job, a project, tasks

— A CEO & CTO sit above project leads, who lead specialist teams

— A lead delegates; the team hands work between each other

— This is my NexaPaw office — 65 agents, live in a 3D HQ

A company of agents — running on your machine.

From crew to office: a Lead delegates

The Lead returns a typed plan; Python dispatches it. You define the staff, not the steps.

```
class Assignment(BaseModel):      # a role must name a real staff member
    role: Literal["CV Engineer", "Backend Engineer", "Frontend Engineer", "QA Engineer"]
    work: str

class WorkPlan(BaseModel):
    assignments: List[Assignment]

# 1. the Lead decides who does what
planning = Task(description=f"Assign this goal to your team ({', '.join(team)}): {brief}",
                expected_output="A WorkPlan with at most 3 assignments", agent=lead,
                output_pydantic=WorkPlan)
plan = Crew(agents=[lead], tasks=[planning]).kickoff().pydantic

# 2. Python dispatches the Lead's decisions to the specialists
tasks = [Task(description=f"{a.work}. Use at most 4 bullet points.", agent=team[a.role],
              expected_output="At most 4 bullet points") for a in plan.assignments]
result = Crew(agents=list(team.values()), tasks=tasks).kickoff()
print(result)
```

→ **Like a NexaPaw project team: the Lead decides who does what; an invalid role fails validation.**

What the office buys you — and costs

- Specialists beat one generalist on complex, multi-part work.
- The manager handles coordination you'd otherwise hard-code.
- But more agents = more model calls = more time and nondeterminism.
- **Local models are slower than hosted —** size the crew to your hardware.
- In my tests, Process.hierarchical never reached a specialist on Llama 3.1 or 3.2. Check who actually ran.

HONEST TRADE-OFF

A virtual office is powerful and harder to control. Reach for it only when a single agent plateaus.

03 Make It Real

From a script to something you can trust

Reliability: guardrails + a human in the loop

Validate outputs; keep consequential actions behind a person.

```python
# guardrail: Python validates the output; failures go back to the agent
def short_update(output: TaskOutput) -> tuple[bool, Any]:
    if len(output.raw.split()) > 120:
        return (False, "Too long. Keep it under 120 words.")
    return (True, output.raw)

status_task = Task(description="...", expected_output="...", agent=writer,
                   guardrail=short_update, guardrail_max_retries=2)

# gate risky actions behind a human — draft, never send
@tool("send_email")
def send_email(to: str, body: str) -> str:
    "Save a draft for human approval; does NOT send."
    Path("outbox.md").write_text(body)
    return "DRAFT saved to outbox.md — needs approval"

# stop a stuck agent from looping forever
planner = Agent(role="Project Planner", goal="...", backstory="...",
                llm=llm, max_iter=8, max_retry_limit=2)
```

→ **Agents can act. Keep irreversible actions — send, pay, delete — behind human approval.**

Observe it, and control cost

- Turn on verbose / logging to see each thought, tool call, and result.
- **Trace tokens and latency —** local models are free but slower.
- Cache repeated calls; keep prompts and passed context tight.
- Evaluate on a fixed set of briefs with known-good outputs before you trust it.

BEFORE PRODUCTION

If you can't measure it on a test set, you can't trust it in front of a stakeholder.

Responsible & safe agents

- **Least privilege —** give an agent only the tools it truly needs.
- Local-first keeps customer data off third-party servers (helps with PDP Law).
- Watch for prompt injection when agents read external text, email, or web pages.
- **Keep a human on any decision about people —** hiring, credit, performance.

SIMPLE RULE

An agent should propose; a person should dispose — for anything that matters.

The whole build, in seven lines

1. An agent is an LLM plus tools, memory, and a loop (ReAct).
2. CrewAI runs that loop; Ollama runs the model locally.
3. Agent + Task + Crew is a working assistant in ~40 lines.
4. Tools are plain Python functions the agent may call.
5. Typed (Pydantic) outputs make agents composable.
6. A Lead + specialists = a virtual office (typed delegation).
7. Guardrails, a human in the loop, and evals make it trustworthy.

Questions & build ideas

Q&A

What project chore would you hand an agent crew first?

- **Which of your tools would it need —** calendar, repo, tracker?
- Where must a human stay in the loop?

Thank you

Questions & Build Ideas

Felix Yustian Setiono · https://linktr.ee/felixyustian · felixyustian@gmail.com

Build Your AI Project Manager — a hands-on MurzEdu session